

Angular Moderno

05 - Controles de Fluxo e Pipes

Controles de Fluxo e Pipes

Aprenda a controlar dinamicamente o que é exibido na tela com controles de fluxo **@if** e **@for**, transforme dados com pipes nativos e crie seu primeiro pipe customizado.



Controles de Fluxo e Pipes

Domine as ferramentas que tornam suas interfaces inteligentes, dinâmicas e muito mais poderosas.

Controles de Fluxo

Controle o que aparece e quando aparece na tela

Pipes

Transforme e formate dados de forma elegante

Pipe Customizado

Crie suas próprias transformações personalizadas

O que você vai aprender

Ao final desta aula, você terá as ferramentas essenciais para construir interfaces dinâmicas com Angular.

Confira o roteiro completo:



Controlar exibição com @if

Mostre ou esconda elementos da tela com base em condições lógicas



Renderizar listas com @for

Percorra arrays e gere HTML dinamicamente para cada item



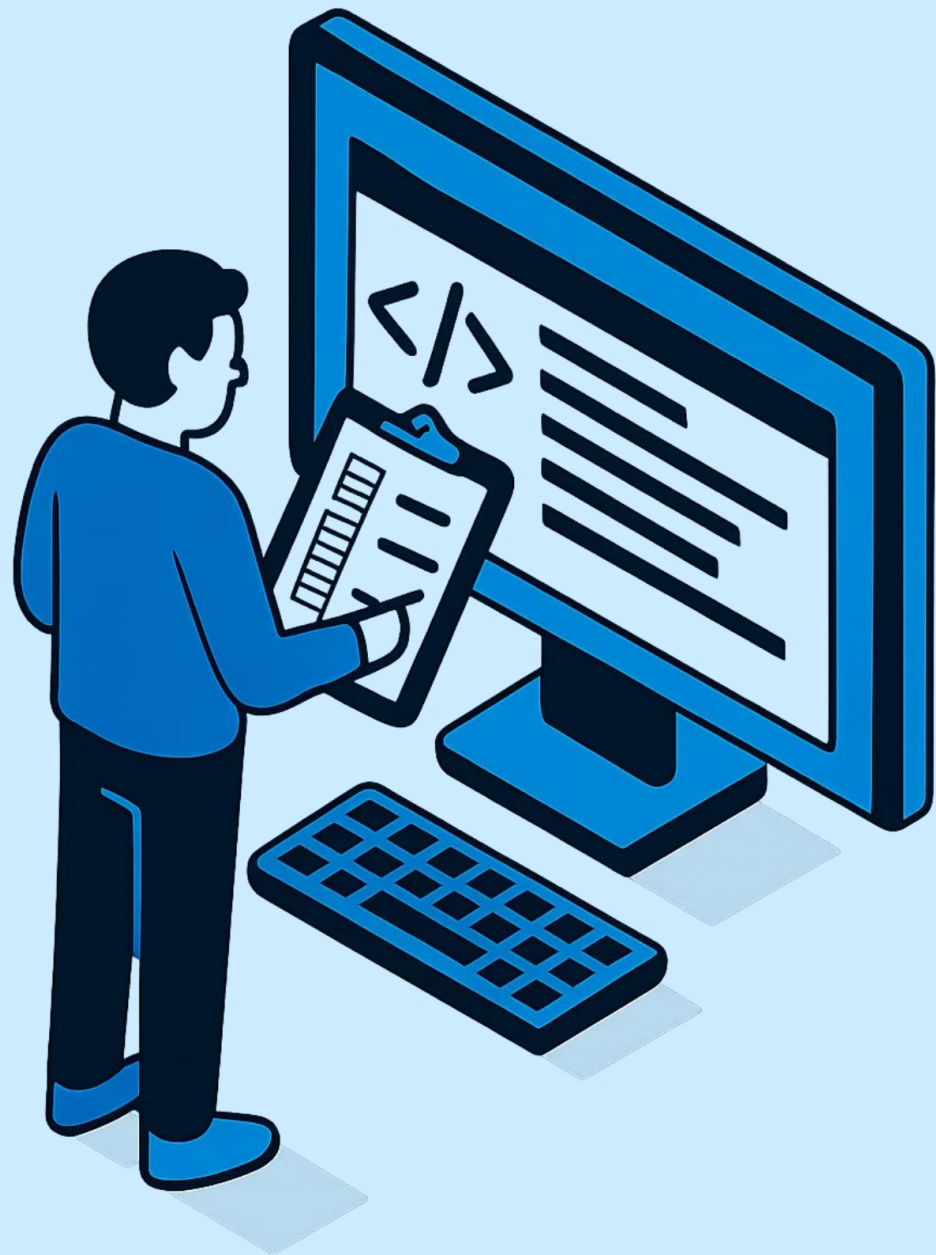
Utilizar pipes para formatar dados

Use pipes nativos como currency, uppercase e date



Criar um pipe customizado

Implemente sua própria lógica de transformação de dados



Antes de Começar

Antes de avançar para o conteúdo desta aula, verifique se o seu ambiente está pronto. Esses três pontos são essenciais:

Projeto anterior
funcionando

O projeto criado
na Aula 4 deve
estar sem erros

Componente
produto criado

O componente
produto deve
existir no projeto

ng serve
rodando

Execute `ng
serve` no
terminal e acesse
`localhost:4200`

 Se algum desses itens ainda não está pronto, volte ao material 04 antes de continuar.

Relembrando a Aula Anterior

O que você já sabe

Na aula anterior, você aprendeu a exibir dados na tela usando interpolação:

```
{{ nome }}
```

Isso mostra o valor de uma variável diretamente no HTML.

Para onde vamos agora

A interpolação simples é poderosa, mas ainda limitada. Agora vamos dar um grande salto: trabalhar com **lógica condicional** e **listas de dados**.

Com controles de fluxo, você vai controlar o *que* aparece, *quando* aparece e *quantas* vezes aparece. Tudo com código limpo e organizado.

O que são Controles de Fluxo?

Controles de Fluxo (Built-in Control Flow) são uma sintaxe nativa e otimizada para manipular a renderização condicional e a repetição de elementos no HTML dos componentes.



O que aparece

Controle quais elementos são renderizados na tela com base em condições



Quantas vezes aparece

Repita blocos de HTML para cada item de uma lista automaticamente



Quando aparece

Mostre ou esconda elementos conforme o estado da sua aplicação muda

Controle de Fluxo no Angular Moderno

Os **controles de fluxo** são um tipo especial de lógica que modifica a **estrutura do DOM**. Elas adicionam, removem ou repetem elementos HTML. Utilizam uma sintaxe baseada em blocos iniciados por @.

As duas mais usadas no dia a dia do Angular são:

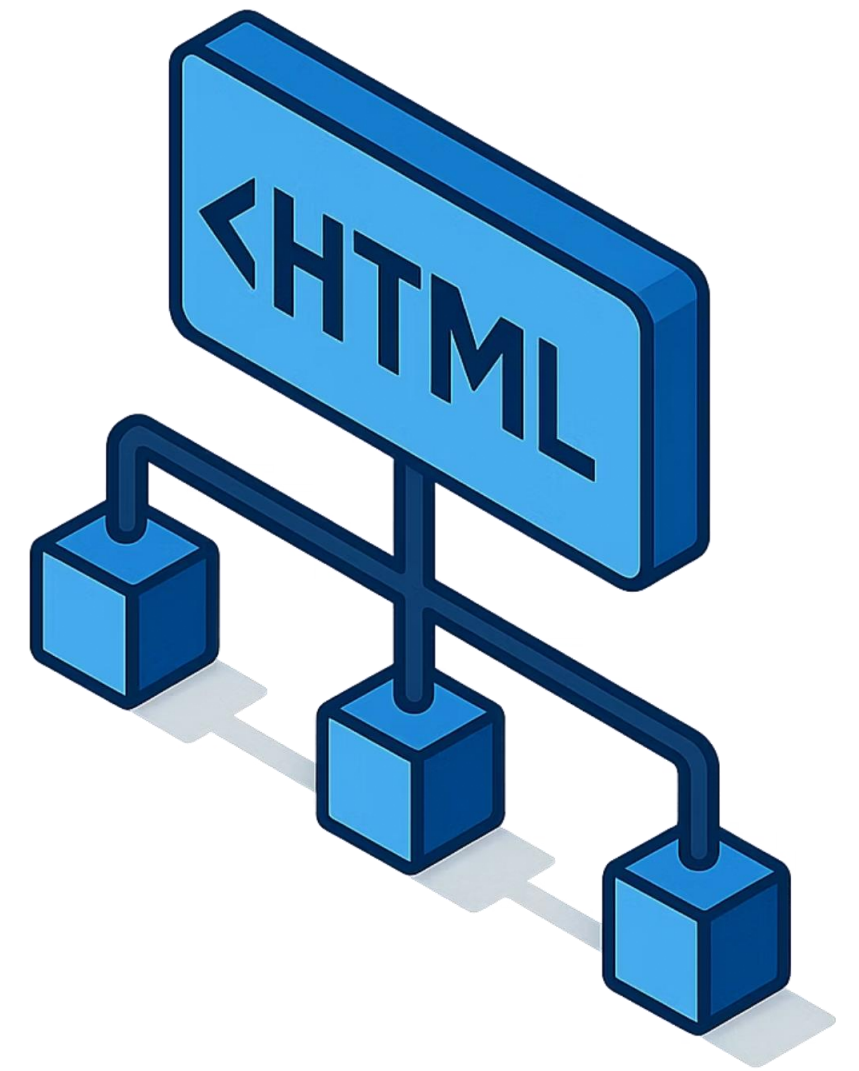
@if

Renderiza um elemento
somente se uma condição
for verdadeira

@for

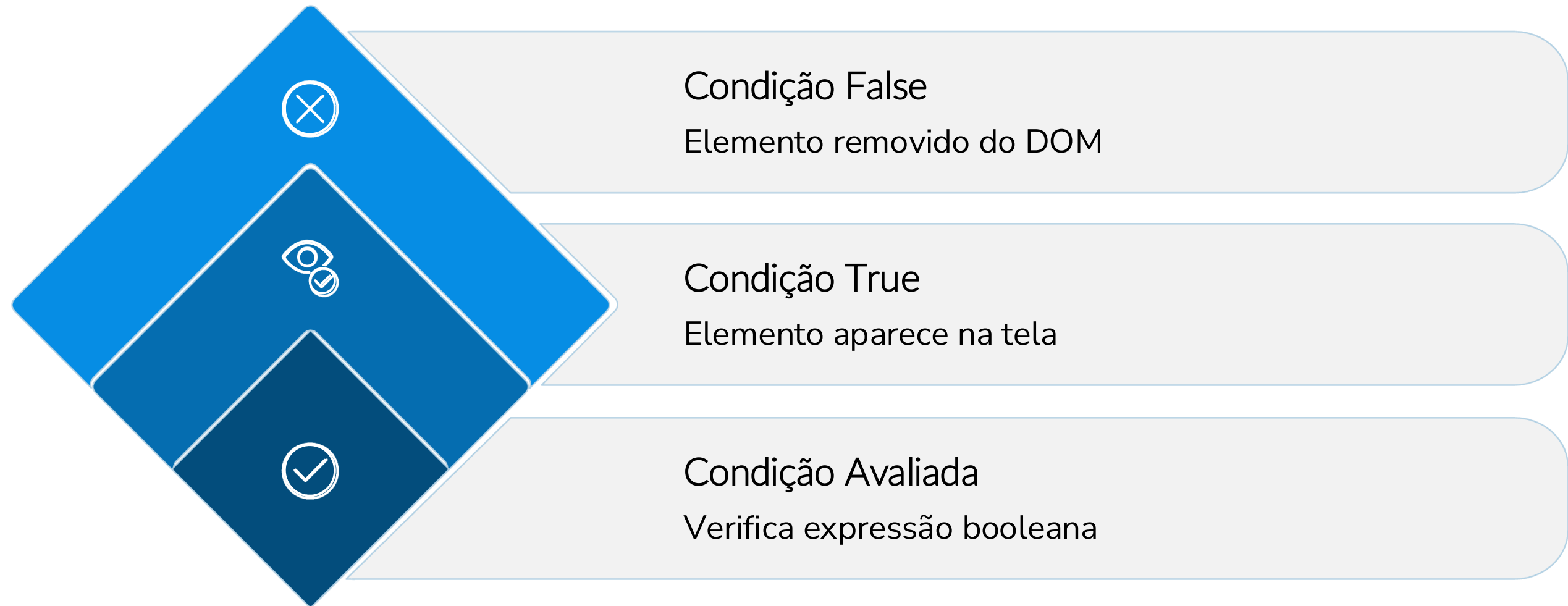
Repete um elemento para
cada item de uma lista

- ❏ O arroba (@) antes do nome do controle de fluxo é a notação que avisa o Angular: "este item vai modificar a estrutura do HTML". Nunca esqueça de colocá-lo!



Introdução ao @if

O **@if** é usado para **exibir ou esconder** elementos do HTML com base em uma condição. Se a condição for **true**, o elemento aparece. Se for **false**, ele é removido completamente do DOM.



Diferente de simplesmente esconder com CSS, o **@if** **remove o elemento do DOM** quando a condição é falsa, o que melhora a performance e a lógica da aplicação.

Exemplo com @if

Veja como implementar o **@if** na prática. Primeiro, defina uma propriedade booleana no componente e depois use-a no template HTML:

produto.ts

```
mostrarPreco = true;
```

Declare a variável de controle no componente
TypeScript

produto.html

Apague o parágrafo `<p>Preço: R$ {{ preco }}</p>`

Insira no mesmo lugar:

```
@if (mostrarPreco) {  
    <p>Preço: R$ {{ preco }}</p>  
}
```

O parágrafo só aparece quando `mostrarPreco` for `true`.

👍 Com apenas uma linha extra no HTML, você adicionou lógica condicional à sua interface!

Testando o @if

Agora experimente alterar o valor da variável e observe o comportamento em tempo real no navegador:

Antes — mostrarPreco = true

```
mostrarPreco = true;
```

O elemento aparece na tela normalmente
com o preço visível.

Depois — mostrarPreco = false

```
mostrarPreco = false;
```

O bloco deixa de ser renderizado.

Salve o arquivo e o navegador irá atualizar automaticamente. Isso é o **hot reload** do Angular funcionando para você!

Por que isso é tão importante?

Com o `@if`, você passa a **controlar a interface com lógica de negócio**, sem precisar escrever JavaScript para manipular o DOM diretamente. Isso é um dos pilares do desenvolvimento moderno com frameworks.

Sem Angular

Selecionar elemento, alterar `style.display`, gerenciar estado manualmente...

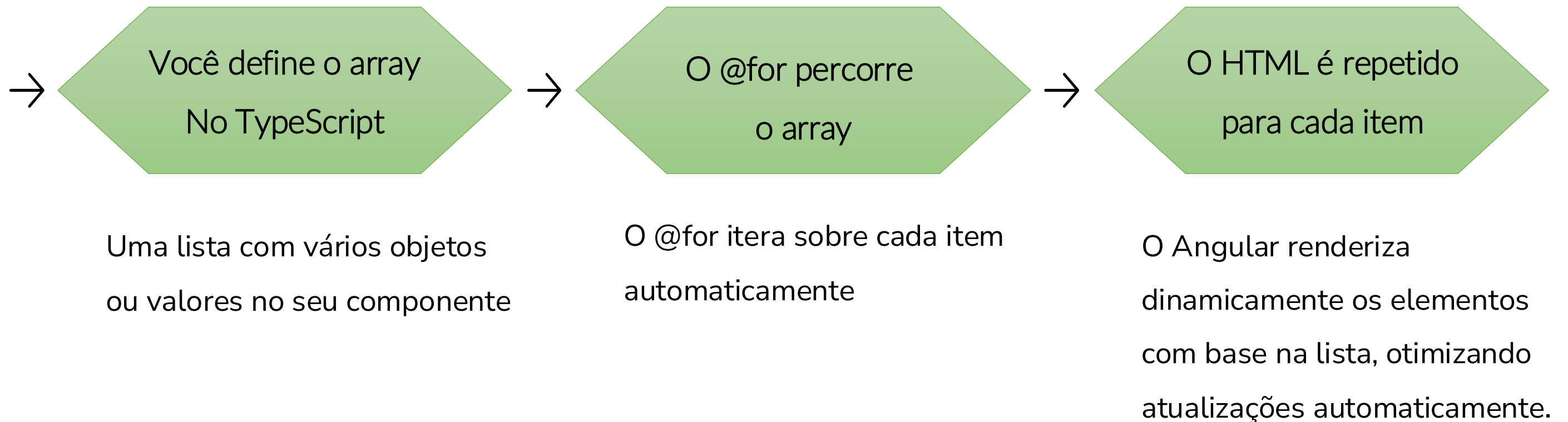
Com `@if`

Uma condição, uma linha!
O Angular cuida de tudo automaticamente.



Introdução ao @for

O **@for** é o controle de fluxo ideal para trabalhar com **listas e arrays**. Ele percorre uma coleção de dados e cria um bloco HTML para cada item de forma automática e eficiente.



Criando uma Lista de Produtos

Primeiro, vamos definir um array de produtos no componente TypeScript. Cada produto é um objeto com nome e preço:

 produto.ts (Ao final da classe Produto)

```
produtos = [  
  { nome: 'Monitor', preco: 1500 },  
  { nome: 'Mouse',    preco: 150  },  
  { nome: 'Teclado',  preco: 250  }  
];
```

O que temos aqui?

Um array chamado `produtos` com **3 objetos**. Cada objeto possui duas propriedades:

- `nome` — string com o nome do produto
- `preco` — number com o valor

Essa é a forma mais comum de estruturar dados em Angular — objetos tipados dentro de arrays.

Usando @for no HTML

Com o array definido, agora basta usar o **@for** no template para renderizar cada produto automaticamente:

 produto.html

```
@for (item of produtos; track item) {  
  <div>  
    <h3>{{ item.nome }}</h3>  
    <p>Preço: R$ {{ item.preco }}</p>  
  </div>  
}
```

Entendendo a sintaxe

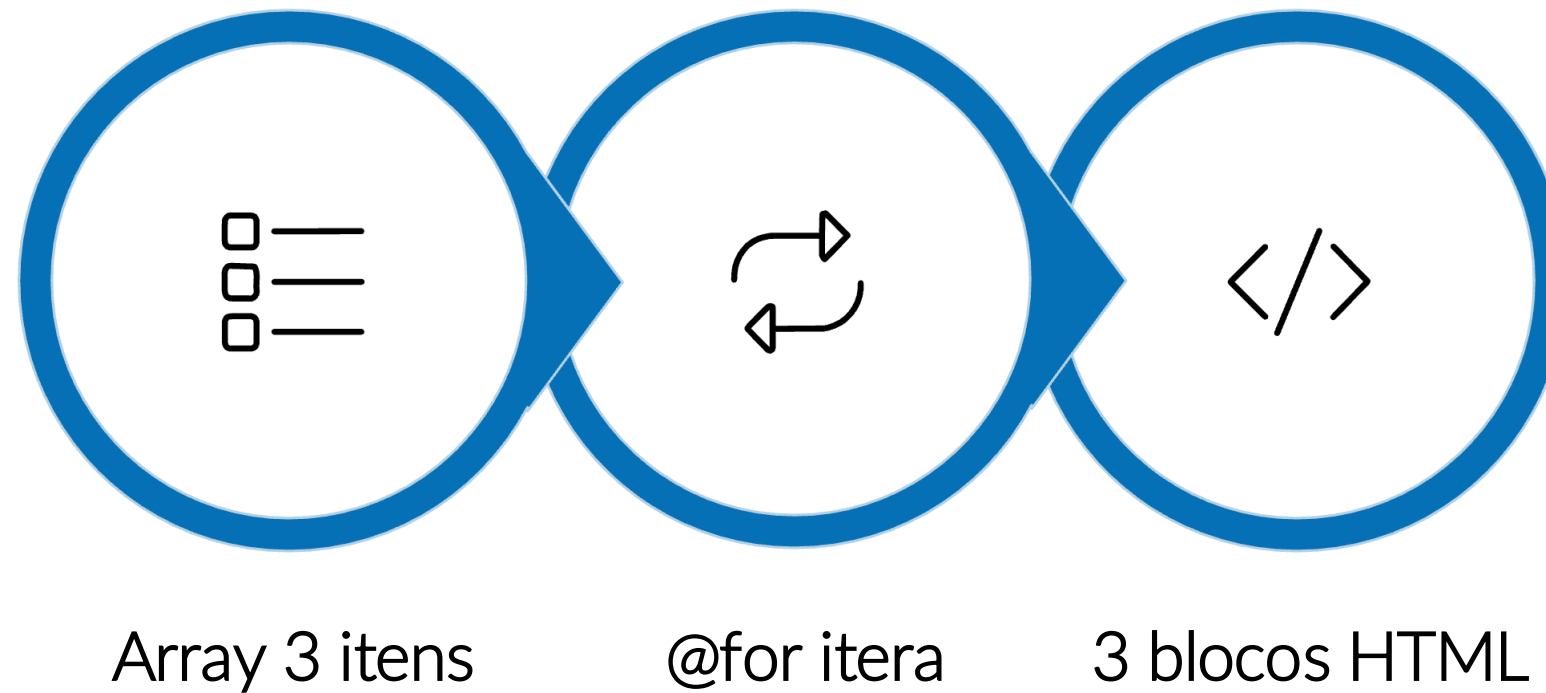
- `item` — cria uma variável local para cada iteração
- `of produtos` — referencia o array do componente
- `track item` — ajuda o Angular a identificar cada elemento da lista e melhorar performance.
- `item.nome` — acessa a propriedade do objeto atual



O `item` pode ter qualquer nome. Use nomes descritivos como `produto`, `usuario`, etc.

Resultado Esperado

Após salvar os arquivos, o Angular processa o @for e gera o HTML automaticamente para cada item do array:



Para um array com 3 produtos, o Angular gera **3 blocos div** idênticos em estrutura, mas com os dados de cada item. Se o array tiver 100 itens, serão 100 blocos, sem alterar uma linha do HTML!

Evoluindo o Projeto

Parabéns! Com @if e @for implementados, seu projeto já tem características de uma **aplicação real**. Veja o que você já conquistou:



Múltiplos
produtos

A interface exibe
todos os itens do
array
automaticamente



Dados dinâmicos

Altere o array e a
tela se atualiza
instantaneamente



Se aproxima de
um e-commerce

Listagem de
produtos é o
coração de
qualquer loja
virtual





ATENÇÃO: Fique atento!

O Angular Moderno NÃO UTILIZA MAIS O RECURSO DE DIRETIVAS.

É muito comum, ao buscar informações pela internet, encontrar soluções Angular que utilizam diretivas como ***ngIf** e ***ngFor**.

Estas diretivas não são mais utilizadas no Angular moderno e já estão sinalizadas como depreciadas (**deprecated**).

As diretivas foram substituídas pelos controles de fluxo.

O que são Pipes?

Pipes são transformadores de dados usados diretamente no template HTML. Eles pegam um valor, aplicam uma transformação e exibem o resultado **sem alterar o dado original** no componente.

A sintaxe é simples: use o símbolo |

Pipes Nativos do Angular

O Angular já vem com vários pipes prontos. Você não precisa criar estas formatações. Basta aplicar no template:

T

uppercase / lowercase

Converte texto para maiúsculas ou minúsculas:

```
{{ nome | uppercase }}
```

\$

currency

Formata números como moeda:

```
{{ preco | currency: 'BRL' }}
```



date

Formata datas de forma legível:

```
{{ data | date: 'dd/MM/yyyy' }}
```

1

number

Formata números com casas decimais:

```
{{ valor | number: '1.2-2' }}
```



Mas Atenção!

O Angular Moderno exige que você importe explicitamente o que usa no componente!

Exemplo de uso de `Currency` e `UpperCase`. Estas alterações precisam ser realizadas no arquivo `.ts` do componente.

```
import { CurrencyPipe, UpperCasePipe } from '@angular/common';

@Component({
  imports: [CurrencyPipe, UpperCasePipe],
})
```

Uso no HTML	Import necessário no .ts
currency	CurrencyPipe
uppercase	UpperCasePipe
lowercase	LowerCasePipe
date	DatePipe
number	DecimalPipe

Usando o Pipe currency

O pipe **currency** é um dos mais usados em aplicações de e-commerce. Ele formata automaticamente um número como valor monetário, incluindo símbolo da moeda e separadores:

Alterações em
`produto.ts`:

```
import { CurrencyPipe } from '@angular/common';

@Component({
  imports: [CurrencyPipe],
})
```

 `produto.html` (modificar o `item.preco` existente)

```
<p>
  Preço: {{ item.preco | currency:'BRL' }}
</p>
```

Resultado na tela

O número bruto `3500` é exibido como:

| **R\$ 3.500,00**

O Angular usa automaticamente as regras de formatação do **locale BRL** (Real Brasileiro): símbolo, separador de milhar e decimais.

Usando o Pipe uppercase

O pipe **uppercase** converte qualquer string para letras maiúsculas. Perfeito para títulos e destaques visuais no HTML:

Alterações em
`produto.ts`:

```
import { CurrencyPipe, UpperCasePipe } from '@angular/common';

@Component({
  imports: [CurrencyPipe, UpperCasePipe],
})
```

 `produto.html`

```
<h2>
  {{ item.nome | uppercase }}
</h2>
```

Resultado na tela

O valor `"Monitor"` é exibido como:

| **MONITOR**

 A variável `item.nome` continua com o valor original `"Notebook"` no TypeScript. O pipe não altera o dado — apenas muda a exibição!

Vantagens dos Pipes

Os pipes não são apenas convenientes. Eles são uma **boa prática de arquitetura**. Veja por que você deve usá-los sempre que precisar transformar dados para exibição:

Código mais limpo

A lógica de formatação fica no template, separada da lógica de negócio do componente

Transformações rápidas

Aplique formatações complexas com uma única palavra no HTML, sem funções extras

Fácil leitura

Qualquer dev entende `{{ preco | currency }}` imediatamente.
É autoexplicativo

Reutilizável

Um pipe criado uma vez pode ser usado em qualquer template do projeto

Criando um Pipe Customizado

Quando os pipes nativos não atendem a uma necessidade específica do seu projeto, você pode criar o seu próprio.

O Angular CLI facilita muito esse processo:

Terminal

```
ng generate pipe pipes/preco-formatado
```

Ou use a forma abreviada:

```
ng g pipe pipes/preco-formatado
```

O que acontece?

O Angular CLI irá automaticamente:

- Criar a pasta `pipes/` se não existir
- Gerar o arquivo `preco-formatado-pipe.ts`
- Criar o arquivo de testes correspondente

Você precisará importar o pipe no componente para utilizá-lo.

O que o Angular Criou?

Após o comando, o Angular CLI gera a estrutura base do pipe. Veja o arquivo criado automaticamente:

preco-formatado.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'precoFormatado',
})
export class PrecoFormatadoPipe implements PipeTransform {
  transform(value: unknown, ...args: unknown[]): unknown {
    return null;
  }
}
```

Entendendo a estrutura

- `@Pipe` — decorator que registra a classe como pipe
- `name` — o nome usado no template HTML
- `transform()` — método onde a lógica é implementada

 Precisamos modificar o método `transform()`!

Implementando o Pipe

Agora vamos adicionar a lógica de formatação personalizada dentro do método `transform()`. O objetivo é exibir preços no formato brasileiro com prefixo "R\$":

 preco-formatado.pipe.ts

```
transform(valor: number): string {  
  return 'R$ ' + valor.toFixed(2);  
}
```

O que cada parte faz

- `valor: number` — o pipe recebe um número como entrada
- `: string` — o retorno é sempre uma string formatada
- `toFixed(2)` — garante sempre 2 casas decimais
- `'R$ ' +` — adiciona o prefixo do Real

✔ Simples, direto e funcional. É assim que pipes customizados funcionam!

Usando o Pipe Customizado no HTML

Com o pipe implementado, basta importar no `.ts` e referenciá-lo pelo `name` definido no decorator:

produto.ts

```
import { PrecoFormatadoPipe } from
'../../pipes/preco-formatado-pipe';

@Component({
  imports: [CurrencyPipe,
    UpperCasePipe, PrecoFormatadoPipe],})
```

produto.html

```
@for (item of produtos; track item) {
  <div>
    <h3>{{ item.nome | uppercase }}</h3>
    <p>Preco: {{ item.preco | precoFormatado }}</p>
  </div>
}
```

- ❏ Lembre-se: o nome do pipe no HTML deve ser idêntico ao valor do atributo `name` no decorator `@Pipe` do arquivo `preco-formatado-pipe.ts`

Resultado

Com o pipe customizado aplicado, os valores são exibidos de forma limpa e padronizada para todos os itens da lista:

NOTEBOOK
R\$ 3500.00

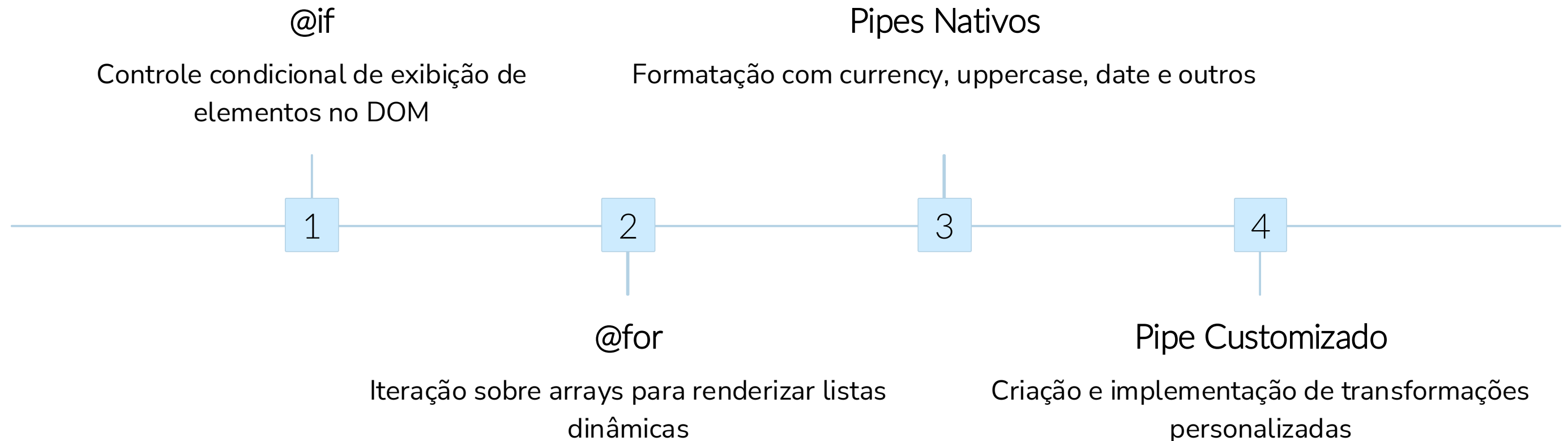
MOUSE
R\$ 150.00

TECLADO
R\$ 250.00

Cada preço passa pelo método `transform()` do pipe e é devolvido formatado. Isso acontece automaticamente para cada item do `@for`, sem nenhuma linha extra de código TypeScript!

O que Aprendemos até Aqui

Esta foi uma aula densa e muito produtiva! Veja o resumo completo do que você dominou:



✔ Você agora tem as ferramentas fundamentais para construir interfaces Angular dinâmicas e bem formatadas!

Boas Práticas

Adotar boas práticas desde o início forma hábitos profissionais e torna seu código mais legível, manutenível e consistente. Siga estas diretrizes:

 Use `@if` para controle de exibição

Sempre que precisar mostrar ou esconder elementos baseado em lógica, prefira o `@if` ao invés de manipular CSS diretamente.

 Use `@for` para listas

Nunca copie e cole blocos HTML manualmente para exibir itens repetidos. O `@for` é mais eficiente, limpo e dinâmico.

 Use pipes para formatação

Toda transformação de exibição deve usar pipes. Não coloque lógica de formatação dentro de métodos no TypeScript quando um pipe resolve de forma mais elegante.

Parabéns!

Seu projeto Angular está evoluindo para algo cada vez mais próximo de uma aplicação real. Revise o que você conquistou hoje:



Listas dinâmicas

Renderizando
coleções com
`@for` de forma
eficiente



Controle de
interface

Exibindo e
escondendo
elementos com
`@if` e lógica real



Transformação
de dados

Formatando
valores com
pipes nativos e
customizados

Continue praticando! Experimente adicionar novos produtos ao array, criar novos pipes e combinar `@if` com `@for` no mesmo componente.

Dúvidas

?

